

Документация проекта GraphRAGon с поддержкой Graph of Thoughts

Utopialvo

30 июня 2026 г.

Содержание

1	Введение	2
2	Архитектура системы	2
3	Модель графа знаний	3
4	Обработка входного текста	4
5	Разрешение конфликтов	5
6	Вопросно-ответная система	6
6.1	Простой режим (без Graph of Thoughts)	6
6.2	Graph of Thoughts	6
7	Хранилище Memgraph	7
8	Особенности реализации и принятые решения	8
9	Заключение	9

1 Введение

Graph RAG (Retrieval-Augmented Generation) — это подход к вопросно-ответным системам, объединяющий поиск релевантной информации в графе знаний и генерацию ответа с помощью большой языковой модели (LLM). В отличие от классических RAG-систем, опирающихся на векторный поиск по коллекции неструктурированных документов, графовая версия хранит извлечённые факты в виде сущностей и связей между ними. Это позволяет моделировать семантические отношения и давать более точные, контекстные ответы. Под графом знаний здесь понимается структура, состоящая из узлов-сущностей и рёбер-отношений, каждое из которых снабжено указанием на исходный текстовый фрагмент (пассаж), обеспечивая прослеживаемость (provenance). Векторные представления (эмбединги) сущностей, полученные с помощью модели sentence-transformers, используются для быстрого поиска семантически близких узлов через встроенный в графовую СУБД Memgraph векторный индекс.

Данный проект реализует полный пайплайн:

- Загрузка произвольного текста, его нарезка на пересекающиеся чанки с использованием библиотеки LangChain.
- Извлечение именованных сущностей и связей из каждого чанка с помощью локальной LLM, подключённой через унифицированный интерфейс LangChain (класс `ChatOpenAI`), с применением структурированного вывода.
- Сохранение сущностей, связей и исходных фрагментов в графовой СУБД Memgraph с управлением транзакциями и потокобезопасным доступом.
- Разрешение конфликтов между противоречащими друг другу отношениями (опциональный этап, запускается отдельным вызовом) с привлечением LLM в роли арбитра.
- Ответ на пользовательский вопрос с использованием техники *Graph of Thoughts* (GoT). Она реализована на базе `StateGraph` из библиотеки LangGraph и включает этапы генерации, оценки, селекции, итеративного улучшения и слияния мыслей. Это позволяет гибко управлять процессом рассуждения и получать многогранные ответы.

Система ориентирована на исследовательские и прототипные задачи, демонстрируя возможность построения полноценного графового RAG на локальном оборудовании с открытыми моделями. Ключевые особенности — кэширование дорогостоящих LLM-запросов через SQLite (встроенный кэш LangChain), кэширование эмбедингов, устойчивость к ошибкам модели, потокобезопасность при параллельной работе, автоматическое создание резервных копий графа при штатном завершении и гибкая настройка всех этапов через единый YAML-конфиг.

2 Архитектура системы

Центральным элементом является класс `GraphRAG`, расположенный в модуле `src/graph_rag.py`. Он координирует работу всех подсистем и предоставляет пользователю высокоуровневый API: `process_text`, `ask`, `resolve_conflicts`, `clear` и другие. Конфигурация

загружается из YAML-файла в датакласс `GraphRAGConfig`, после чего инициализируются необходимые компоненты.

Взаимодействие с языковой моделью построено на базе класса `ChatOpenAI` из библиотеки `langchain_openai`. Он принимает URL локального сервера, имя модели и параметры генерации (температуру, максимальное число повторных попыток). Кэширование LLM-запросов обеспечивается глобальным SQLite-кэшем, который подключается через вызов `set_llm_cache(SQLiteCache(database_path=...))`. Таким образом, все повторные запросы с идентичными промптами, моделью и температурой обслуживаются мгновенно.

Эмбединги вычисляются с помощью класса `HuggingFaceEmbeddings` из `langchain_huggingface`, который оборачивает модель `sentence-transformers`. Модель загружается один раз при создании экземпляра и используется как для вычисления эмбедингов сущностей и пассажей, так и для векторного поиска. Параметры точности (`float32`) и размера батча задаются в конфигурации.

Хранилище графа реализовано в модуле `src/memgraph_store.py` классом `MemgraphStore`. Оно использует официальный Neo4j-драйвер для взаимодействия с Memgraph через протокол Bolt. Все операции с графом защищены рекурсивной блокировкой (`threading.RLock`), что делает хранилище потокобезопасным. Модуль предоставляет контекстный менеджер `transaction()` для выполнения набора операций в одной транзакции с автоматическим коммитом или откатом.

Извлечение сущностей и отношений выполняется классами `LLMEntityExtractor` и `RelationExtractor`, которые используют один и тот же LLM-клиент. Оба построены на цепочках `LangChain`: промпт формируется через `ChatPromptTemplate`, а ответ парсится с помощью метода `with_structured_output`, который на основе Pydantic-моделей извлекает JSON прямо из ответа модели. Это исключает необходимость ручного разбора и повышает надёжность.

Разрешение конфликтов вынесено в отдельный модуль `src/conflict_resolver.py` и выполняется классом `ConflictResolver`. Он также использует структурированный вывод и работает итеративно для каждой пары сущностей.

История диалогов ведётся простым менеджером `HistoryManager`, основанным на циклической очереди фиксированного размера. В текущей версии история не подставляется в промпты, но доступна для внешнего анализа.

Модуль `src/got_graph.py` содержит класс `GoTGraph`, реализующий Graph of Thoughts на базе `StateGraph` из `LangGraph`. Он конфигурируется теми же параметрами из YAML и предоставляет метод `run(question, context)`, который строит и выполняет граф операций над мыслями.

Все компоненты слабо связаны: каждый класс отвечает за свою задачу и взаимодействует через вызовы методов с чёткими сигнатурами. Это упрощает тестирование, замену отдельных частей и расширение функциональности.

3 Модель графа знаний

Граф в Memgraph организован как совокупность узлов и связей нескольких типов, образующих фактический и мета-уровень.

Сущность (`Entity`) — узел с меткой `:Entity`. Он обладает свойствами `name` (строка, уникальный идентификатор), `type` (тип сущности, например `PERSON`, `LOCATION`, `OBJECT`, `ACTION`, `TIME` и др.) и `embedding` — векторное представление имени сущности, вычисленное моделью эмбедингов. Тип сущности одновременно является именем

узла схемы (`:Schema`), с которым сущность связывается отношением `:INSTANCE_OF`. Это позволяет впоследствии фильтровать сущности по типу и строить онтологии.

Пассаж (Passage) — узел `:Passage` с уникальным идентификатором `id` и полным текстом чанка в свойстве `text`. Пассаж представляет исходный фрагмент текста, из которого были извлечены сущности и отношения. Связь `:MENTIONED_IN` соединяет сущность с пассажем, указывая на её упоминание. Она создаётся с помощью `MERGE`, что гарантирует отсутствие дубликатов даже при многократной обработке одного и того же фрагмента.

Отношение (Relation) — ребро с меткой `:RELATION` между двумя узлами сущностей. Оно имеет свойство `type`, хранящее глагольную фразу (например, "рубил", "находился в"), и свойство `based_on` — список идентификаторов пассажей, из которых это отношение было извлечено. Такая структура позволяет хранить несколько доказательств одного факта: если одно и то же отношение извлекается из разных чанков, их идентификаторы накапливаются в `based_on`. При добавлении отношения используется условная логика в Cypher-запросе: при создании нового ребра список инициализируется текущим пассажем, а при совпадении — проверяется, нет ли уже такого идентификатора, и если нет, он добавляется. Это гарантирует идемпотентность и чистоту данных.

Схема (Schema) — узлы с меткой `:Schema`, имеющие свойства `name` и `type`. Они создаются автоматически при первом появлении нового типа сущности или отношения и служат для самодокументирования графа.

Векторный индекс строится на основе свойства `embedding` узлов `Entity`. Для этого используется встроенная в Memgraph процедура `vector_search.search`, опирающаяся на библиотеку USearch. При инициализации хранилища размерность индекса определяется по тестовому эмбедингу, и если существующий индекс не соответствует новой размерности, он автоматически удаляется и создаётся заново. Это позволяет безболезненно менять модель эмбедингов.

4 Обработка входного текста

Метод `process_text` принимает строку текста и опциональный параметр `source`, возвращая список идентификаторов созданных пассажей. Рассмотрим его работу поэтапно.

Для разбиения текста на чанки используется `RecursiveCharacterTextSplitter` из `LangChain`. Он режет текст с учётом иерархии разделителей (абзацы, строки, пробелы, символы), стараясь сохранить семантические границы. Размер чанка и перекрытие задаются в конфигурации (по умолчанию 1000 и 200 символов соответственно). Перекрытие необходимо, чтобы не терять связи между сущностями, оказавшимися на границе разбиения.

Далее для каждого чанка выполняется транзакционная вставка данных. Временный кэш эмбедингов включается перед обработкой всех чанков и отключается после. Благодаря ему одинаковые имена сущностей не приводят к повторным обращениям к модели эмбедингов, что значительно ускоряет загрузку больших текстов.

Внутри транзакции для каждого чанка:

1. Создаётся узел пассажа с уникальным идентификатором вида `<source>_<idx>` и его текстом.

2. Вызывается `LLMEntityExtractor.extract(chunk)`. Он формирует системный промпт с просьбой извлечь сущности в виде JSON-массива объектов `{ "text": "...", "type": "..."}.` Благодаря использованию `with_structured_output` на основе Pydantic-модели `EntitiesList`, ответ модели автоматически преобразуется в список объектов `Entity`. При неудаче (ошибка валидации, сетевой сбой) выполняется до двух повторных попыток.
3. Каждая извлечённая сущность добавляется в граф через `MemgraphStore.add_entity`. Если сущность с таким именем уже существует, при необходимости обновляется её тип (с `UNKNOWN` на более конкретный), и создаётся связь `MENTIONED_IN` с текущим пассажем. Для новой сущности вычисляется эмбединг, и дополнительно создаётся связь со схемой.
4. Затем `RelationExtractor.extract` получает список найденных сущностей (в виде строк) и текст чанка. Аналогичным образом с помощью структурированного вывода модель возвращает список объектов `Relation` с полями `head`, `relation`, `tail`. Каждое отношение валидируется и добавляется в граф. При необходимости автоматически создаются отсутствующие сущности (с типом `UNKNOWN`).
5. Если все операции выполнены успешно, транзакция коммитится. При любом исключении она откатывается, и чанк пропускается. Это обеспечивает атомарность: граф не оказывается в промежуточном несогласованном состоянии.

Метрики обработки (число сущностей, отношений, пассажиров и затраченное время) накапливаются в атрибуте `metrics` экземпляра `GraphRAG`.

5 Разрешение конфликтов

Конфликты между отношениями возникают, когда из разных фрагментов текста для одной и той же пары сущностей извлекаются противоречивые утверждения. Например, «Иван купил молоко» и «Иван продал молоко» — оба не могут быть истинны одновременно. Хранение таких отношений в графе без разрешения ухудшит качество ответов. Для решения этой проблемы предназначен класс `ConflictResolver`, запускаемый вручную методом `resolve_conflicts(dry_run=False)`.

Алгоритм начинается с того, что извлекает все отношения вместе с идентификаторами пассажиров, на основе которых они построены. Отношения группируются по паре (`head`, `tail`). Для каждой группы, где больше одного отношения, запускается итеративный процесс попарного сравнения.

На каждой итерации строится рабочий список активных отношений. Для каждой пары с разными названиями вызывается LLM-арбитр с промптом, в котором перечислены обе тройки и тексты первых связанных пассажиров. Модель должна вернуть JSON с полями: `conflict` (`true/false`), `resolution` (одно из `"keep_first"`, `"keep_second"`, `"merge"`), `new_relation` (для слияния) и `explanation`. Для надёжности используется структурированный вывод в Pydantic-модель `ConflictResolution`, а при неудаче — до двух повторных попыток.

Если конфликт подтверждён, применяется соответствующее действие:

- `"keep_first"` — второе отношение удаляется из графа и из рабочего списка.

- `"keep_second"` — удаляется первое.
- `"merge"` — оба исходных отношения удаляются; создаётся новое с именем, предложенным моделью (или «связан» по умолчанию). В его список `based_on` попадают идентификаторы пассажей обоих старых отношений, что сохраняет полную доказательную базу.

После любого изменения рабочий список перестраивается, и итерация повторяется до тех пор, пока не останется ни одного конфликта или не будет достигнут лимит итераций. Такой подход исключает ошибки, связанные с удалением уже удалённых элементов, и гарантирует, что каждое решение принимается с учётом предыдущих изменений.

Метод возвращает словарь с числом обнаруженных и разрешённых конфликтов, а также их детальным списком. Режим `dry_run=True` позволяет проанализировать конфликты без изменения графа.

6 Вопросно-ответная система

Основной метод `ask(question, use_got=None)` принимает вопрос пользователя и опциональный флаг, определяющий, использовать ли Graph of Thoughts. Если флаг не задан, решение принимается на основе конфигурации (`got_enabled`).

В обоих режимах первым шагом является сбор контекста. Пользовательский вопрос преобразуется в эмбединг, после чего выполняется векторный поиск ближайших сущностей и пассажей (по отдельности). Для каждой найденной сущности извлекаются связанные отношения вместе с текстами пассажей. Если в конфигурации задана глубина случайного обхода (`random_walk_depth > 0`), дополнительно собираются соседние сущности путём обхода отношений на заданную глубину с ограничением количества шагов. Все собранные фрагменты текста объединяются, и, при необходимости, обрезаются до `max_context_length` символов. Если релевантная информация не найдена, контекстом становится строка «Нет релевантной информации».

6.1 Простой режим (без Graph of Thoughts)

При `use_got=False` (или если GoT отключён) система формирует прямой промпт, содержащий контекст и вопрос, и отправляет его LLM. Ответ модели возвращается пользователю. Никакой дополнительной обработки не производится.

6.2 Graph of Thoughts

При включённом GoT контекст и вопрос передаются экземпляру `GoTGraph`, который строит и выполняет граф операций над мыслями. В основе реализации лежит `StateGraph` из библиотеки `LangGraph`. Состояние графа описывается типизированным словарём `GoTState` и включает текущий вопрос, контекст, список мыслей, их оценки, счётчик итераций и финальный ответ.

Граф состоит из нескольких узлов-операций:

1. **Генерация (Generate)**. Создаётся `got_num_thoughts` (по умолчанию 3) мыслей. Для каждой мысли выбирается роль из списка `got_roles` (Аналитик, Критик, Стратег, Скептик, Учёный). Выбор может быть случайным или последовательным, в зависимости от флага `got_use_random_roles`. Роли снабжены подробными описаниями, которые подставляются в системный промпт, заставляя модель рассуждать в определённом стиле. Это вносит необходимое разнообразие. Генерация идёт последовательно для каждой мысли, с температурой `got_generation_temperature`.
2. **Оценка (Score)**. Каждая мысль оценивается LLM по шкале от 0 до 10. Промпт просит модель вернуть только число. Для извлечения числа из ответа используется регулярное выражение. При неудаче оценка считается нулевой. Оценка сохраняется в состоянии. Если флаг `got_score_enabled` снят, всем мыслям присваивается одинаковый балл (5.0).
3. **Селекция (Select)** (опционально, управляется `got_use_select`). Среди полученных мыслей отбираются `got_select_top_k` с наивысшими оценками. Остальные отбрасываются. Это позволяет сфокусироваться на наиболее перспективных вариантах.
4. **Улучшение (Refine)**. Каждая отобранная мысль отправляется на доработку. Промпт просит модель улучшить ответ: устранить неточности, добавить важные детали из контекста, улучшить структуру. Температура задаётся параметром `got_refine_temperature`. После улучшения может снова следовать оценка и селекция (если они включены), образуя цикл. Количество таких циклов задаётся параметром `got_iterations` (по умолчанию 1). Таким образом, при значении 2 мысли пройдут через улучшение дважды, с промежуточной оценкой и отбором.
5. **Слияние (Merge)**. Если мыслей больше одной и флаг `got_merge_enabled` установлен, все оставшиеся мысли объединяются в одном промпте, и модель синтезирует единый связный ответ. В противном случае возвращается первая (или единственная) мысль.

Управление потоком выполнения реализовано через условные рёбра: после оценки проверяется, достигнуто ли максимальное число итераций. Если да — переход на слияние, иначе — на улучшение. После улучшения всегда следует возврат на оценку. Если селекция отключена, соответствующий узел пропускается, и переход идёт напрямую от оценки к развилке. Благодаря такому подходу граф вычислений полностью описывается в `_build_graph` и легко модифицируется.

После выполнения графа итоговый ответ сохраняется в истории диалога.

7 Хранилище Memgraph

Класс `MemgraphStore` инкапсулирует всю работу с графовой СУБД. Подключение осуществляется по протоколу Bolt с использованием официального Neo4j-драйвера для Python. Аутентификация не используется (Memgraph по умолчанию принимает соединения без пароля).

Все публичные методы защищены одним рекурсивным замком (`threading.RLock`), который захватывается перед выполнением любого запроса. Это гарантирует, что даже при одновременных вызовах из разных потоков запросы не будут чередоваться и нарушать целостность данных. Для прототипа, где основные задержки связаны с LLM, а не с графом, такое решение вполне оправдано.

Метод `transaction()` возвращает контекстный менеджер, внутри которого можно выполнять несколько операций в одной транзакции. При выходе из блока без исключений транзакция автоматически коммитится, иначе откатывается. Это особенно важно при вставке чанка, чтобы избежать частичных данных.

Векторный поиск реализован через вызов хранимой процедуры `vector_search.search`. Она принимает имя индекса (`entity_embeddings` или `passage_embeddings`), число ближайших соседей и вектор запроса. Процедура возвращает узлы и косинусное расстояние до них. На основе этих данных формируется список словарей с именем, типом и оценкой релевантности.

Управление жизненным циклом данных включает методы `clear()` и `snapshot()`. `clear` удаляет все узлы и связи графа командой `MATCH (n) DETACH DELETE n`, сбрасывает векторные индексы и создаёт их заново (в блоке `finally`), что позволяет быстро начать новый эксперимент. `snapshot` создаёт резервную копию состояния Memgraph на диске с помощью команды `CREATE SNAPSHOT`. Он вызывается автоматически при вызове `close()` у экземпляра `GraphRAG`, а также при штатном завершении процесса благодаря регистрации через `atexit`.

8 Особенности реализации и принятые решения

В проекте сознательно принят ряд компромиссов между простотой, надёжностью и производительностью, которые делают его удобным для исследовательских задач.

Потокобезопасность. Единая блокировка в хранилище сериализует доступ к графу, а выполнение GoT происходит последовательно в одном потоке. Это исключает гонки и необходимость сложной координации. Для сценариев с интенсивным параллельным чтением это может стать узким местом, но текущие задачи (обработка текста и последовательные вопросы) не предъявляют высоких требований к конкурентности на уровне графа.

Graceful shutdown. Все активные экземпляры `GraphRAG` регистрируются в глобальном списке. При завершении интерпретатора (нормальном или из-за необработанного исключения) вызывается функция, которая для каждого экземпляра пытается создать снейпшот Memgraph и закрыть драйвер. Это предотвращает потерю данных даже при аварийной остановке (например, по `Ctrl+C`), хотя полную гарантию для `kill -9` дать нельзя.

Устойчивость к ошибкам модели. Извлечение сущностей и отношений опирается на структурированный вывод `LangChain`, который внутри использует вызовы API с функциями. Если модель не поддерживает такой режим, это может вызывать ошибку, но для совместимых серверов (в частности, `vllm`, `text-generation-webui`, `llama.cpp`) работает стабильно. Предусмотрены повторные попытки с экспоненциальной задержкой при сетевых сбоях. В случае полного провала экстракции возвращается пустой список, и чанк пропускается, не нарушая целостность графа.

Кэширование. LLM-запросы кэшируются глобально через `SQLiteCache` из `LangChain`. Ключом является MD5-хеш от комбинации промпта, модели и температуры. Это означает, что изменение любого из этих параметров приведёт к промаху кэша. Кэш

эмбедингов действует аналогично на уровне отдельного экземпляра: внутри батча при включённом режиме `enable_batch_cache` все уникальные тексты кодируются только один раз. Старые записи LLM-кэша могут автоматически удаляться по таймауту (`cache_cleanup_hours`), что предотвращает неконтролируемый рост базы.

Атомарность чанка. Каждый чанк — это транзакционная единица: либо весь его набор сущностей и отношений попадает в граф, либо не попадает ничего. Это страхует от ситуаций, когда часть данных сохранена, а часть потеряна из-за сбоя, что могло бы привести к несогласованности.

Ролевые модели в GoT. Использование нескольких чётко описанных ролей (аналитик, критик и т.д.) — простой, но эффективный способ получить разноплановые первоначальные мысли. Роли задаются в конфигурации, их можно менять или дополнять без перекомпиляции кода. Случайный выбор ролей (по умолчанию) вносит дополнительную вариативность.

Гибкость графа операций. Благодаря LangGraph структура рассуждения не зашита жёстко: узлы и переходы можно менять в методе `_build_graph`. В частности, можно убрать оценку и селекцию, зациклить улучшение несколько раз или добавить параллельные ветки генерации. Это открывает широкие возможности для экспериментов с различными стратегиями.

Конфигурируемость. Все параметры — от хоста Memgraph до температур GoT — вынесены в единственный YAML-файл, что позволяет быстро адаптировать систему под новое окружение или модель без правки кода.

9 Заключение

GraphRAGon представляет собой компактную, но функциональную реализацию графового RAG с продвинутой вопросно-ответной подсистемой на основе Graph of Thoughts. Он использует современные библиотеки LangChain и LangGraph, что упрощает интеграцию с LLM и реализацию сложных цепочек рассуждений. Модульная архитектура и детальная конфигурация позволяют использовать проект как основу для экспериментов в области извлечения знаний, построения графов и генерации ответов с учётом многостороннего анализа.

Особое внимание уделено надёжности: транзакционная загрузка, кэширование, безопасное завершение и обработка ошибок делают систему пригодной для повседневной исследовательской работы. Несмотря на прототипный статус, проект демонстрирует, что даже на локальной машине с открытыми моделями можно построить RAG-систему, которая «понимает» текст на уровне фактов и способна рассуждать, а не просто копировать куски контекста.